

Tutorial - Week 05 5COSC019W – Object Oriented Programming – Java

Abstract Class and Interfaces

20 October - 21 October

Abstract Class and its subclasses

- A class that is declared using the “abstract” keyword is known as an abstract class.
- It may or may not include abstract methods, which means in the abstract class, you can have **concrete** methods (methods with implementation) as well as **abstract** methods (without an implementation, without braces, and followed by a semicolon).
- An abstract class can not be instantiated (you are not allowed to create an object of an Abstract class).

1) Download the following Java project from BB – Tutorial Week 05:

https://learning.westminster.ac.uk/ultra/courses/_97941_1/outline/edit/document/_4971539_1?courseId=_97941_1&view=content

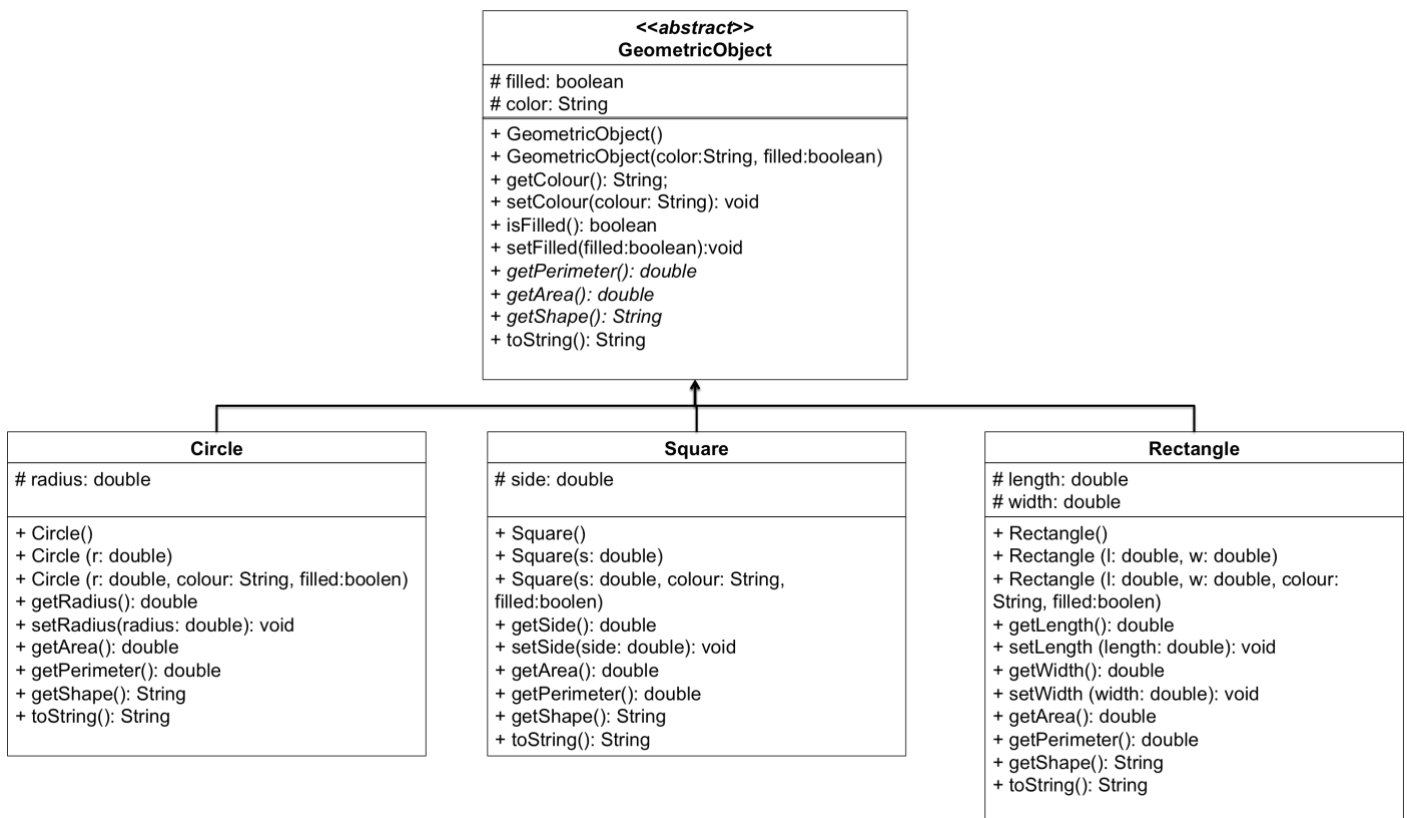
2) Unzip the file and open the project in NetBeans.

3) You will have to modify or edit this project based on the task below.

Task 01

Consider the following diagram that refers to the project you just opened.

We have a superclass `GeometricObject` and its subclasses `Circle`, `Rectangle` and `Square` (`Square` will be added in the next task).



GeometricObject Class

The `GeometricShape` class is defined as an abstract class which contains:

- Two protected instance variables `colour: String` and `filled: boolean`. The protected variable can be accessed by its subclasses and classes in the same package. They are defined with a ‘#’ sign in the class diagram.

- Getter and setter methods for the instance variables and the method `toString()` .
- Three Abstract methods:
 - o *getArea()*: return the area of the geometric object
 - o *getPerimeter()*: return the perimeter of the geometric object
 - o *getShape()*: return a String that indicates the shape of the object (e.g. "Circle" if the object is a circle, "Rectangle" if the object is a rectangle" and "Square" if it is a square.

These methods are shown in italics in the class diagram since they are abstract! The subclasses shall override these abstract methods, providing the proper implementation.

```
public abstract class GeometricObject {
    protected String colour;
    protected boolean filled;

    public GeometricObject(){...}

    public GeometricObject(String colour, boolean filled){...}

    public void setColour(String colour){...}

    public String getColour(){...}

    public void setFilled(boolean filled){...}

    public boolean getFilled(){...}

    public String toString {...}

    public abstract double getArea();

    public abstract double getPerimeter();

    public abstract String getShape();
}
```

Abstract methods with no implementation

Circle Class

The `Circle` class is a subclass of `GeometricObject`. It inherits all the instance variables and the methods of the class `GeometricObject`. `Circle` also has an instance variable `radius`.

Task 02

In the class `Circle` override the following methods:

- 1) `getShape()` – return a String that indicates the shape of the object (e.g. "Circle")
- 2) `getArea()` – return the area of a circle, which is $= \text{PI} * \text{radius} * \text{radius}$
- 3) `getPerimeter()` – return the perimeter (circumference) of a circle, which is $= 2 * \text{PI} * \text{radius}$.

Rectangle Class

`Rectangle` class is a subclass of `GeometricObject`. It inherits all the instance variables and the methods of the class `GeometricObject`. A `rectangle` also has two other instance variables: `length` and `width`.

Task 03

In this class, you should override the following methods:

- 1) `getShape()` – return a String that indicates the shape of the object (e.g. "Rectangle")
- 2) `getArea()` – return the area of a rectangle, which is $= \text{length} * \text{width}$
- 3) `getPerimeter()` – return the perimeter of a rectangle, which is $= 2 * \text{length} + 2 * \text{width}$.

Task 04

Add a class called "Square" to the package *tutorial05_shapecollection* and implement the class Square following the specifications of the class diagram.

Square Class

Square class is a subclass of *GeometricObject*.

It inherits all the instance variables and the methods of the class *GeometricObject*. Square also has another instance variable: *side*.

Add three constructors as represented in the UML diagram.

Add set and get methods for the instance variable.

In this class, you will override the methods *getShape()*, *getArea()* and *getPerimeter()* as you did for the class *Circle*, and *Rectangle*, calculating the area and the perimeter of a square.

Remember that the area of a square is $= \text{side} * \text{side}$, and the perimeter is $= 4 * \text{side}$.

Interface

The interface looks like a class, but it is not a class. An interface can have methods and variables just like the class, but the methods declared in the interface are abstract by default (only method signature, no body). Also, the variables declared in an interface are public, static & final by default.

In the downloaded project, there is an interface called *ShapeCollection*:

```
public interface ShapeCollection {  
  
    public abstract void addShape(GeometricObject shape);  
    public abstract void printShapeList();  
    public abstract boolean runMenu();  
}
```

Task 05 (For this task, you can use the guidance provided below)

Add a new class called *GeometricShapeCollection* that implements the interface *ShapeCollection*.

GeometricShapeCollection has a list of a maximum (*maxNumShape*) *GeometricObjects*. The class should display a menu from where the user can decide if adding a shape (a circle, a square or a rectangle) to this list or printing the list of shapes with the relative area and perimeter.

Hint:

This class implements the interface, so we need to declare the class as follows:

```
public class GeometricShapeCollection implements ShapeCollection {  
    ...  
}
```

The class *GeometricShapeCollection* contains:

- two instance variables:
 - 1) an array of *Array* of *GeometricObject*
 - 2) an integer *maxNumShape* that represents the maximum number of *GeometricObject* that we want to keep in the list
 - 3) an integer (static variable) *totalNumShape* that holds the current name of *GeometricObject*

```
private GeometricObject[] shapeList;
private int maxNumShape;
public static int totalNumShape = 0;
```

- a constructor that takes as a parameter the number of objects we want in the list:

```
public GeometricShapeCollection(int listLength){
    this.maxNumShape = listLength;
    shapeList = new GeometricObject[maxNumShape];
}
```

- the implementation of the abstract methods that are declared in the interface:

- o **public void addShape(GeometricObject shape)- add the shape passed as a parameter to the shapeList**

```
public void addShape(GeometricObject shape){

    // check if there are spaces and add the shape to the list
    if (totalNumShape < maxNumShape){
        shapeList[totalNumShape] = shape;
        totalNumShape++;
    }
    else{
        System.out.println("No more space in the list");
    }
}
```

- o **public void printShapeList() – print all the GeometriObjects saved in the array. It prints the type of shape, the area and the perimeter**

```
public void printShapeList(){

    for (int i = 0; i < totalNumShape; i++) {
        if (shapeList[i].getShape().equals("Circle")) {
            System.out.println("Shape = Circle, Area = " +
                shapeList[i].getArea() + ", Perimeter = " + shapeList[i].getPerimeter());
        }
        // continue the implementation of this method and write here the code to
        // verify if it is a rectangle or a square and print the area and the
        // perimeter

        // could you used instanceof instead of .equals("Circle") to verify the
        // shape?
    }
}
```

Verify if it is a Circle

- o **public boolean runMenu() – users can interact with the system through a console menu, where they add a new shape with all the attributes, print the list of shapes (using the methods just implemented) and exit.**

```
public boolean runMenu(){

    boolean exit = false;

    System.out.println("To Add a new shape press 1");
    System.out.println("To print the list of the shapes press 2");
    System.out.println("To exit press 3");

    Scanner s = new Scanner (System.in);
    int choice = s.nextInt();
```

Console menu from where the user can select one of the three options

```

switch(choice){
    case 1:
        //check if there is more space in the system to add a new geometric
        object
        if (GeometricShapeCollection.totalNumShape < maxNumShape) {

            System.out.println("Press 1 if you want to add a Circle");
            System.out.println("Press 2 if you want to add a Rectangle");
            System.out.println("Press 3 if you want to add a Square");

            int choice2 = s.nextInt();
            s.nextLine();

            System.out.println("Enter the color of your shape");
            String colour = s.nextLine();

            System.out.println("Is it filled? (y/n)");
            String isFilled = s.nextLine();
            boolean filled = false;

            if (isFilled.equals("y")) {
                filled = true;
            } else if (isFilled.equals("n")) {
                filled = false;
            } else {
                filled = false; // false by default - you
            }
            // improve your code adding validation on user input

            switch (choice2){
                case 1:
                    // it is a circle
                    System.out.println("Insert the radius");
                    int radius = s.nextInt();
                    Circle c = new Circle(radius, colour, filled);
                    this.addShape(c);
                    break;

                case 2:
                    // write here the code if the rectangle is selected

                case 3:
                    // write here the code if the square is selected
            }
            break;

            case 2:
                this.printShapeList();
                break;

            case 3:
                exit = true;
                break;
        }
    }

    return exit;
}

```

The user can select if add one of the three shapes

These properties (colour and filled) are common to all shapes!

can

If the circle is selected

The other two initial options: print the list or exit

- the implementation of the main where the menu is launched:

```
public static void main(String[] args){  
  
    // create a parking  
    ShapeCollection sys = new GeometricShapeCollection(3);  
    boolean exit = false;  
  
    while (!exit){  
        exit = sys.runMenu();  
    }  
}
```



The menu is displayed until the user decides to exit

Some theory questions in preparation for the In-class test:

1) Explain the following:

- Class;
- Object;
- Class variable;
- Method overloading;
- Constructor.

2) What is the difference between a public field and a protected field?

3) Using a UML class diagram, draw an example (you can invent a typical scenario) for each of the following inter-class relationships.

- composition
- aggregation
- inheritance

Something more: Typical Interview Questions/Exercises

Try to solve the following exercise. It is a typical question that is likely to be in an interview test. You will get the solution next week on BB!

1) Implement an algorithm to check if a string has all unique characters. Consider that you cannot use additional data structures.